

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Inner Classes

Total Questions : 22

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Inner class

Questions Question - 1

```
class MyOuter {  
    class MyInner { }  
}
```

after compile how many classes will be created ?

- 1.MyOuter.class and MyOuter\$MyInner.class
- 2.MyOuter\$MyInner.class
- 3.MyOuter.class
- 4.None of the above

Explanation :

A is the correct answer.

two class files will be created. MyOuter.class MyOuter\$MyInner.class

Question - 2

What is the output for the below code ?

```
public abstract class A {  
    public void printValue() {  
        System.out.println("A");  
    }  
}
```

```
1. public class Test{  
2.     public static void main (String[] args){  
3.         A a1 = new A() {  
4.             };  
5.             a1.printValue();  
6.     }  
7. }
```

- 1.Compilation fails due to an error on line 3
- 2.A
- 3.Compilation fails due to an error on line 5
- 4.null

Explanation :

B is the correct answer.

The A a1 reference variable refers not to an instance of abstract class A, but to an instance of an anonymous (unnamed) subclass of A. So no compilation error.

Question - 3

Which statement is true?

- 1.an inner class instance can never stand alone without a direct relationship to an instance of the outer class
- 2.you can access the inner class is through a live instance of the outer class
- 3.Inner class can access private member of the outer class.
- 4.All of the above

Explanation :

D is the correct answer.

an inner class instance can never stand alone without a direct relationship to an instance of the outer class. you can access the inner class is through a live instance of the outer class. Inner class can access private member of the outer class.

Question - 4

What is the output for the below code ?

```
public class Outer {
    private int a = 7;

    class Inner {
        public void displayValue() {
            System.out.println("Value of a is " + a);
        }
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        Outer mo = new Outer();
        Outer.Inner inner = mo.new Inner();
        inner.displayValue();
    }
}
```

- 1.Value of a is 7
- 2.Compilation Fails

3.Compilation succeed but Runtime Exception

4.Value of a is 0

Explanation :

A is the correct answer.

an inner class instance can never stand alone without a direct relationship to an instance of the outer class. you can access the inner class is through a live instance of the outer class. Inner class can access private member of the outer class.

Question - 5

Modifiers Applied to Inner Classes is _____?

1.private

2.final

3.public

4.static

Explanation :

A, B, C and D is the correct answer.

Modifiers Applied to Inner Classes is final abstract public private protected static strictfp

Question - 6

What is the output for the below code ?

```
class Outer {
    private String x = "Outer variable";
    void doStuff() {
        String z = "local variable";
        class Inner {
            public void seeOuter() {
                System.out.println("Outer x is " + x);
                System.out.println("Local variable z is " + z);
            }
        }
    }
}
```

1.Compile Fail : local variable z is can't access from within inner class

2.Compilation succeed but Runtime Exception

3.Compile without error.

4.None of the above

Explanation :

A is the correct answer.

the inner class object cannot use the local variables of the method the inner class is in.
only final can be accessible.

Question - 7

What is the output for the below code ?

```
class Outer {
    private String x = "Outer variable";
    void doStuff() {
        final String z = "local variable";
        class Inner {
            public void seeOuter() {
                System.out.println("Outer x is " + x);
                System.out.println("Local variable z is " + z);
            }
        }
        Inner mi = new Inner();
        mi.seeOuter();
    }

    public static void main(String... args){
        Outer out = new Outer();
        out.doStuff();
    }
}
```

What is the output?

1. Compile Fail : local variable z is can't access from within inner class
2. Outer x is Outer variable Local variable z is local variable
3. Outer x is Outer variable
4. Compilation succeed but Runtime Exception

Explanation :

B is the correct answer.

the inner class object cannot use the local variables of the method the inner class is in.
only final can be accessible.

Question - 8

What is the output for the below code ?

```
public class Tech {
    public void tech() {
        System.out.println("Tech");
    }
}
```

```

}

public class Atech {

    Tech a = new Tech() {
        public void nontech () {
            System.out.println("anonymous nontech tech");
        }
        public void tech() {
            System.out.println("anonymous tech");
        }
    };

    public void dothis() {
        a.tech();
        a.nontech();
    }

    public static void main(String... args){
        Atech atech = new Atech();
        atech.dothis();
    }
}

```

- 1.anonymous nontech tech anonymous tech
- 2.anonymous tech
- 3.Compilation fails
- 4.Compilation succeed but Runtime Exception

Explanation :

C is the correct answer.

anonymous subclass of the specified class type. Compile fail because Tech class does not have nontech() method. compiler will complain if you try to invoke any method on an anonymous inner class reference that is not in the superclass class definition.

Question - 9

What is the output for the below code ?

```

public class Tech {
    public void tech() {
        System.out.println("Tech");
    }
}

public class Atech {

    Tech a = new Tech() {

```

```

        public void tech() {
            System.out.println("anonymous tech");
        }
    };

    public void dothis() {
        a.tech();
    }

    public static void main(String... args){
        Atech atech = new Atech();
        atech.dothis();
    }
}

```

- 1.anonymous tech
- 2.Tech
- 3.Compilation fails
- 4.None of the above

Explanation :

A is the correct answer.

This is anonymous subclass of the specified class type. anonymous inner class (anonymous subclass) overridden the Tech super class of tech() method. So Subclass method will get called.

Question - 10

What is the output for the below code ?

```

public interface Ianimal {
    public void eat();
}

public class Tiger {

    Ianimal c = new Ianimal() {
        public void eat() {
            System.out.println("Tiger eat non-veg");
        }
    };

    public void doit(){
        c.eat();
    }

    public static void main(String... args){
        Tiger t = new Tiger();
        t.doit();
    }
}

```

```
    }  
}
```

- 1.Tiger eat non-veg
- 2.Compilation succeed but Runtime Exception
- 3.Compile fail : Interface can't Instantiate
- 4.None of the above

Explanation :

A is the correct answer.

This is anonymous implementer of the specified interface type. Interface is implemented by anonymous class and output is "Tiger eat non-veg".

Question - 11

Is the bellow code compile without error?

```
public class ThreadTest {  
  
    Runnable r = new Runnable() {  
        public void run() { }  
    };  
  
}
```

- 1.compile without error
- 2.compile with error because Runnable is an Interface.
- 3.Compilation succeed but Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

instantiating an implementer of the Runnable interface (an anonymous implementation class). So compile without error.

Question - 12

What is the true about the bellow code

```
class Outer {  
    static class Inner { }  
}
```

- 1.Inner class can be accessed without having an instance of the outer class.
- 2.Inner class can't be accessed without having an instance of the outer class.

- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

Static Inner class can be accessed without having an instance of the outer class. This is like other static members.

Question - 13

What is the output for the below code ?

```
class Outer {
    static class Inner {
        void go() {
            System.out.println("Inner.go");
        }
    }
}

public class Test {
    static class Inner1 {
        void go() {
            System.out.println("Inner1.go");
        }
    }

    public static void main(String... args) throws Exception {
        Outer.Inner bn = new Outer.Inner();
        bn.go();
        Inner1 in = new Inner1();
        in.go();
    }
}
```

- 1.Compile fail because Inner1 in = new Inner1(); Inner class can't be accessed without having an instance of the outer class.
- 2.Inner.go Inner1.go
- 3.Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

Static Inner class can be accessed without having an instance of the outer class. This is like other static members.

Question - 14

Which are true about an anonymous inner class?

- 1.can extend exactly one class and implement exactly one interface
- 2.can extend exactly one class or implement exactly one interface.
- 3.can implement multiple interfaces if it does not extend a class.
- 4.None of the above

Explanation :

B is the correct answer.

syntax of an anonymous inner class allows for only one. can extend exactly one class or implement exactly one interface. Ianimal c = new Ianimal() { } Ianimal can be class or interface.

Question - 15

What will be the result of compiling and run the following code:

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        Object o = new Object() {  
            public boolean equals(Object obj) {  
                return true;  
            }  
        }  
        System.out.println(o.equals("test"));  
    }  
}
```

- 1.Compile Fail: semicolon missing at Object o = new Object() { public boolean equals(Object obj) { return true; } };
- 2.test
- 3.Compilation succeed but Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

semicolon missing at Object o = new Object() { public boolean equals(Object obj) { return true; } };

Question - 16

What is the output for the below code ?

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        class Inner {  
            public String name;  
            public Inner(String s) {  
                name = s;  
            }  
        }  
  
        Inner in = new Inner("test");  
        System.out.println(in.name);  
  
    }  
}
```

- 1.test
- 2.An exception occurs at runtime
- 3.Compilation Fails.
- 4.None of the above

Explanation :

A is the correct answer.

This is just Inner class call.

Question - 17

Which are true about a static nested class?

- 1.static nested class can be accessed without having an instance of the outer class.
- 2.static nested class can't be accessed without having an instance of the outer class.
- 3.It does not have access to non-static members of the enclosing class.
- 4.None of the above

Explanation :

A and C is the correct answer.

static nested class can be accessed without having an instance of the outer class. If the outer class is named Outer, and the nested class is named Inner, it can be instantiated using new Outer.Inner(); It does not have access to non-static members of the enclosing class.

Question - 18

Which are true about method local inner class?

1. method local inner class can be abstract.
2. It can access private members of the enclosing class.
3. A method-local inner class can be instantiated only within the method where the inner class is defined.
4. All of the above

Explanation :

A and D is the correct answer.

All statements are true.

Question - 19

```
class A {  
    A(String s) {  
    }  
    A() {  
    }  
}  
  
1. class B extends A {  
2.     B() { }  
3.     B(String s) {  
4.         super(s);  
5.     }  
6.     void test() {  
7.         // insert code here  
8.     }  
9. }
```

Which of the below code can be insert at line 7 to make clean compilation ?

1. A a = new B();
2. A a = new B(5);
3. A a = new A(String s);
4. All of the above

Explanation :

A is the correct answer.

A a = new B(); is correct because anonymous inner classes are no different from any other class when it comes to polymorphism.

Question - 20

What is the output for the below code ?

```
interface A {  
    public void printValue();  
}  
  
1. public class Test{  
2.     public static void main (String[] args){  
3.         A a1 = new A() {  
4.             public void printValue() {  
5.                 System.out.println("A");  
6.             }  
7.         };  
8.         a1.printValue();  
9.     }  
10. }
```

- 1.Compilation fails due to an error on line 3
- 2.A
- 3.Compilation fails due to an error on line 8
- 4.null

Explanation :

B is the correct answer.

The A a1 reference variable refers not to an instance of interface A, but to an instance of an anonymous (unnamed) class. So no compilation error.

Question - 21

What is the output for the below code ?

```
class A {  
    class A1 {  
        void printValue() {  
            System.out.println("A.A1");  
        }  
    }  
}  
  
1. public class Test{  
2.     public static void main (String[] args){  
3.         A.A1 a1 = new A.A1();  
4.         a1.printValue();  
5.     }  
6. }
```

- 1.Compilation fails due to an error on line 3
- 2.A.A1
- 3.Compilation fails due to an error on line 4

4.null

Explanation :

A is the correct answer.

No enclosing instance of type A is accessible. Must qualify the allocation with an enclosing instance of type A (e.g. x.new A() where x is an instance of A). You have to do like : A a = new A(); A.A1 a1 = a.new A1();

Question - 22

```
class A {  
    class A1 {  
        void printValue() {  
            System.out.println("A.A1");  
        }  
    }  
}  
  
1. public class Test{  
2.     public static void main (String[] args){  
3.         A a = new A();  
4.         // INSERT CODE  
5.         a1.printValue();  
6.     }  
7. }
```

Which of the below code inserted at line 4, compile and produce the output "A.A1"?

- 1.A.A1 a1 = new A.A1();
- 2.A.A1 a1 = a.new A1();
- 3.A a1 = new A.A1();
- 4.All of the above

Explanation :

B is the correct answer.

correct inner class instantiation syntax is A a = new A(); A.A1 a1 = a.new A1();